

MediGuide AI

A Mini Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

Aarchi Israni

EN23CS301011

Aarushi Nahata

EN23CS301014

Aarya Saraf

EN23CS301017

Under the Guidance of

Prof. Amrata Gupta

Prof Roshni Verma



Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work “**MediGuide AI**” is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn therein; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner

Prof Amrata Gupta

Prof Roshni Verma

Assistant Professor

Medicaps University

Declaration

We hereby declare that the project entitled “**MediGuide AI**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, completed under the supervision of Roshni **Verma and Amrata Gupta, Assistant Professor, Computer Science and Engineering**, Faculty of Engineering, Medicaps University Indore, is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Certificate

We, **Prof Roshni Verma and Prof Amrata Gupta** certify that the project entitled “**MediGuide AI**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Aarchi Israni, Aarushi Nahata and Aarya Saraf**, is the record carried out by them under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof Amrata Gupta

Prof Roshni Verma

Department of Computer science and engineering

Medicaps University, Indore

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Mediacaps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. (Dr.) Ratnesh Litoriya**, Associate Dean, Faculty of Computing Engineering, Mediacaps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Aarchi Israni

Aarushi Nahata

Aarya Saraf

B.Tech. III Year (A)

Department of Computer Science & Engineering

Faculty of Engineering

Mediacaps University, Indore

Abstract

Healthcare awareness and early medical guidance are essential to prevent serious diseases and improve quality of life. Many individuals ignore early symptoms or lack clarity in choosing the right specialist, leading to delayed treatment and complications.

The project **MediGuide AI – AI-Based Preventive Healthcare Platform** addresses these issues by providing an integrated web-based solution for health risk prediction, doctor recommendation, and online consultation. The system predicts possible health risks for the next three years, classifies them into Low, Medium, and High categories, and provides preventive suggestions such as diet improvement, exercise, stress management, and regular checkups.

The platform is developed using modern technologies like React.js / Next.js for frontend, Node.js and Express.js for backend, MongoDB for database, and OpenAI API / Gemini API for AI-based predictions.

The report is organized into chapters covering introduction, requirements, system design, implementation and testing, results, and conclusions with future scope.

Keywords: Artificial Intelligence, Healthcare Platform, Preventive Healthcare, Risk Prediction, Doctor Recommendation, Web Application, MongoDB, Node.js, React.js, Telemedicine.

Table of Contents

		Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	vii
	List of figures	ix
	Abbreviations	x
	Notations & Symbols	xi
Chapter 1	Introduction (Whichever is applicable)	
	1.1 Introduction	
	1.2 Literature Review	1
	1.3 Objectives	2
	1.4 Significance	2
	1.5 Research Design	2
	1.6 Source of Data	3
	1.7 Chapter Scheme	3
Chapter 2	REQUIREMENTS SPECIFICATION	
	2.1 User Characteristics	4
	2.2 Functional Requirements	4
	2.3 Dependencies	5
	2.4 Performance Requirements	5
	2.5 Hardware Requirements	5
	2.6 Constraints & Assumptions	6
Chapter 3	DESIGN (Whichever is applicable)	
	3.1 Algorithm (if Applicable)	7
	3.2 Function Oriented Design for procedural approach	8
	3.3 System Design (Whichever is applicable)	
	3.3.1 Data Flow Diagrams (Level 0, Level1)	10

	3.3.2 Activity Diagram	11
	3.3.3 Flow Chart	12
	3.3.4 Class Diagram	13
	3.3.5 ER Diagram	14
	3.3.6 Sequence diagram	15
	3.4 Database Design	
	3.4.1 Logical Database Design	16
	3.4.2 Physical Database Design	17
Chapter 4	Implementation, Testing, and Maintenance	
	4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation	19
	4.2 Testing Techniques and Test Plans (According to project)	20
	4.3 Installation Instructions	21
	4.4 End User Instructions	22
Chapter 5	Results and Discussions	
	5.1 User Interface Representation (of Respective Project)	24
	5.2 Brief Description of Various Modules of the system	24
	5.3 Snapshots of system with brief detail of each	24
	5.4 Back Ends Representation (Database to be used)	27
	5.5 Snapshots of Database Tables with brief description	28
Chapter 6	Summary and Conclusions	29
Chapter 7	Future scope	31
	Appendix	31
	Bibliography	31

List of Figures

Figure 3.3.1	Data Flow Diagram level 0
Figure 3.3.2	Data Flow Diagram level 1
Figure 3.3.3	Activity Diagram
Figure 3.3.4	Flow Chart
Figure 3.3.5	Class Diagram
Figure 3.3.6	ER Diagram
Figure 3.3.7	Sequence Diagram

Abbreviations

The following abbreviations are used in the MediGuide AI project report:

- **AI** – Artificial Intelligence
- **API** – Application Programming Interface
- **UI** – User Interface
- **UX** – User Experience
- **DBMS** – Database Management System
- **NoSQL** – Not Only Structured Query Language
- **EHR** – Electronic Health Records
- **ML** – Machine Learning
- **JWT** – JSON Web Token
- **HTTP** – HyperText Transfer Protocol
- **HTTPS** – HyperText Transfer Protocol Secure
- **IDE** – Integrated Development Environment
- **CRUD** – Create, Read, Update, Delete
- **JSON** – JavaScript Object Notation
- **REST** – Representational State Transfer
- **URL** – Uniform Resource Locator
- **OTP** – One Time Password

Notations & Symbols

The following notations and symbols are used in the project:

- $\rightarrow$: Represents flow of process or data
- $\leftrightarrow$: Represents two-way communication
- $\{\}$: Represents a set or collection of data
- $[]$: Represents a list or array
- $()$: Represents function or method call
- $=$: Assignment operator
- $==$: Equality comparison
- $!=$: Not equal to
- $\%$: Percentage (used in risk prediction)
- **PK** : Primary Key (unique identifier in database)
- **FK** : Foreign Key (reference to another entity)
- **ID** : Identifier (unique value)
- **n** : Number of records or users
- **O()** : Time complexity notation

Chapter-1

1.1 Introduction

The rapid growth of digital technology has significantly transformed various sectors, including healthcare. However, many patients still face challenges in identifying the appropriate medical specialist for their symptoms, accessing timely consultations, and understanding potential future health risks. Healthcare awareness and early risk detection play a vital role in preventing serious medical conditions. With advancements in Artificial Intelligence (AI), healthcare systems can now assist patients by providing predictive insights and preventive guidance based on health and lifestyle data.

MediGuide is an AI-assisted web application designed to help users understand their potential future health risks and connect with the right medical specialists at an early stage. The system collects patient information such as age, symptoms, existing diseases, and lifestyle factors, and uses AI models to generate preventive health insights. It also provides doctor recommendations and a basic online consultation feature.

The platform aims to bridge the gap between patients and healthcare providers by combining symptom analysis, doctor matching, online consultation, and predictive analytics into a single integrated system. By doing so, MediGuide enhances accessibility, improves healthcare decision-making, and encourages proactive health management.

1.2 Literature Review

The integration of Artificial Intelligence (AI) in healthcare has significantly improved early disease detection, preventive care, and decision support systems. Several studies and technological advancements have contributed to the development of AI-assisted health platforms similar to MediGuide.

AI-based healthcare systems gained major attention after the development of deep learning models capable of analyzing medical data. Predictive healthcare models are now widely used for risk assessment of chronic diseases such as diabetes, heart disease, and hypertension.

Modern AI language models such as OpenAI's GPT and Google's Gemini have enabled conversational health assistants capable of analyzing user-provided data and generating informative responses. Several healthcare applications like symptom checkers and telemedicine platforms provide basic guidance and doctor consultation services. However, many of these systems either focus only on symptom checking or only on consultation. MediGuide integrates predictive analysis, preventive suggestions, and doctor recommendation in a single platform.

1.3 Objectives

The main objectives of MediGuide are:

- To collect patient health data in an interactive manner
- To analyze patient symptoms, existing conditions, and lifestyle factors
- To predict possible future health risks for the next 3 years using AI
- To recommend appropriate medical specialists based on risks and symptoms
- To enable simple online consultation with doctors
- To promote preventive healthcare and early medical intervention

1.4 Significance

The significance of MediGuide lies in promoting preventive healthcare and improving accessibility to medical guidance. Many individuals delay consulting healthcare professionals due to lack of awareness or confusion regarding symptoms and specialists. MediGuide helps users understand health risks early and guides them toward the right doctors.

It also supports better healthcare decision-making through AI-driven insights, encourages early intervention, and reduces chances of severe health complications.

1.5 Research Design

The research design of MediGuide is based on a system development approach involving data collection, AI analysis, recommendation systems, and web-based implementation.

- **Frontend Layer:** Developed using React / Next.js and Tailwind CSS
- **Backend Layer:** Developed using Node.js and Express
- **Database:** MongoDB for temporary storage
- **AI Integration:** OpenAI or Gemini API for predictive health analysis

The methodology includes collecting user data, symptom analysis, disease correlation, doctor recommendation, risk prediction, and dashboard visualization.

1.6 Source of Data

The data used in MediGuide is collected from both primary and secondary sources.

Primary Sources:

- Patient inputs such as age, symptoms, lifestyle habits, health history

Secondary Sources:

- Medical datasets
- Research papers
- WHO reports
- AI documentation and healthcare literature

1.7 Chapter Scheme

The project report is divided into the following chapters:

- **Chapter 1:** Introduction, Literature Review, Objectives, Significance, Research Design, Source of Data
- **Chapter 2:** System Analysis and Problem Definition
- **Chapter 3:** Methodology and System Architecture
- **Chapter 4:** Implementation and Working Procedure
- **Chapter 5:** Results, Conclusion, and Future Scope

This structure ensures a logical flow from understanding the problem to presenting the solution and outcomes

Chapter-2

REQUIREMENTS SPECIFICATION

2.1 User Characteristics

The MediGuide AI system is designed for the following user classes:

- **Patients:** Primary users of the system who need a simple and user-friendly interface for symptom entry, viewing AI predictions, doctor recommendations, and consultation booking. Users may have basic technical knowledge.
- **Doctors:** Medical professionals with moderate technical skills who review patient details, provide chat-based consultations, and manage appointments.
- **Administrators:** Responsible for maintaining the system, managing user and doctor accounts, monitoring activity, configuring APIs, and database supervision.
- **Future Users:** Healthcare organizations that may use the system later for analytics, patient engagement, and hospital integration.

2.2 Functional Requirements

The MediGuide AI system shall provide the following features:

- User registration, login, password recovery, and secure authentication.
- Patient health data collection including age, symptoms, medical history, and lifestyle factors.
- AI-based symptom analysis and predictive health risk assessment.
- 5-year risk probability classification as Low / Medium / High.
- Preventive health suggestions based on risk results.
- Doctor recommendation according to symptoms and risk level.
- Appointment booking, rescheduling, and cancellation.
- Secure online consultation through chat.
- Patient dashboard to view profile, history, reports, and previous consultations.
- Admin panel for managing users, doctors, and monitoring system operations.

2.3 Dependencies

The system depends on the following external technologies and services:

- AI APIs such as OpenAI or Gemini for health prediction and recommendations.
- MongoDB database for storing user data, history, and doctor records.
- Cloud server or dedicated hosting environment.
- Stable internet connection for users and server communication.
- Modern browsers such as Chrome, Edge, Firefox, Safari.
- Optional services like email (SMTP) for verification and password reset.
- Future integration with hospital systems or EHR platforms.

2.4 Performance Requirements

The system should meet the following performance standards:

- Home page loading time should be within **3 seconds**.
- Login and registration response time should be within **2 seconds**.
- AI symptom analysis and prediction should respond within **5–8 seconds**.
- Database queries should complete within **2 seconds**.
- Client-side validation should occur within **1 second**.
- Support at least **100 concurrent users**.
- Backend should handle at least **50 API requests per second**.
- System availability should be **99% uptime** with minimal downtime.

2.5 Hardware Requirements

Client Side

- Desktop / Laptop / Tablet / Smartphone
- Minimum processor speed: **1.5 GHz**
- Minimum **4 GB RAM**
- Stable 4G or broadband internet

- Modern web browser support

Server Side

- Cloud server or dedicated server
- Multi-core processor
- Minimum **8 GB RAM**
- Sufficient storage for database and backups
- Secure network connectivity

2.6 Constraints & Assumptions

Constraints

- The platform provides preventive guidance only and is **not a diagnostic medical tool**.
- AI predictions depend on third-party API availability and response quality.
- User data security, privacy compliance, and consent must be maintained.
- Development time is limited due to academic project schedule.
- Performance depends on network bandwidth and server load.

Assumptions

- Users have internet access and basic digital literacy.
- Users provide accurate health information.
- AI APIs and database services remain operational.
- Doctors registered on the platform are verified professionals.
- Users understand that results are for awareness and guidance purposes only.

Chapter-3

DESIGN

3.1 Algorithm

The MediGuide AI project uses an AI-assisted prediction and recommendation algorithm to analyze user health data and provide preventive guidance.

Algorithm: Health Risk Prediction and Doctor Recommendation

Step 1: User Registration/Login

- User creates account or logs into the system securely.

Step 2: Data Collection

- Collect user inputs such as age, gender, symptoms, medical history, lifestyle habits, stress level, sleep pattern, and activity level.

Step 3: Data Validation

- Check whether all required fields are filled correctly.
- If invalid, display error message.

Step 4: AI Processing

- Send structured patient data to AI module (OpenAI / Gemini API).
- AI analyzes symptoms and health factors.

Step 5: Risk Prediction

- Predict possible future health risks for the next 3 years.
- Calculate risk percentage and classify risk level as:
 - Low
 - Medium
 - High

Step 6: Preventive Suggestions

- Generate lifestyle recommendations such as exercise, diet, sleep improvement, stress control, and medical checkups.

Step 7: Doctor Recommendation

- Match symptoms/risk with suitable specialist doctors from database.
- Show doctor name, specialization, experience, and availability.

Step 8: Output Display

- Show prediction results on dashboard using charts/cards.

Step 9: Consultation Support

- Allow booking or chat consultation with doctor.

Step 10: Store Records

- Save prediction history, consultation details, and reports in database.

3.2 Function Oriented Design (Procedural Approach)

The MediGuide AI system follows a modular function-oriented design where each function performs a specific task.

Main Modules / Functions

1. Authentication Module

Functions:

- registerUser()
- loginUser()
- logoutUser()
- resetPassword()

2. User Input Module

Functions:

- collectPatientData()
- validateInput()
- updateProfile()

3. AI Prediction Module

Functions:

- sendToAI()
- analyzeSymptoms()
- predictRisk()
- classifyRiskLevel()
- generateSuggestions()

4. Doctor Recommendation Module

Functions:

- findSpecialist()
- fetchDoctorList()
- displayDoctors()

5. Consultation Module

Functions:

- bookAppointment()
- startChat()
- saveConsultationHistory()

6. Dashboard Module

Functions:

- showPredictionHistory()
- showReports()
- visualizeRiskChart()

7. Database Module

Functions:

- saveUserData()
- fetchUserData()
- updateRecords()
- deleteRecords()

8. Admin Module

Functions:

- manageUsers()
- manageDoctors()
- monitorSystem()

3.3 System Design

3.3.1 Data Flow Diagram (Level 0, Level 1)

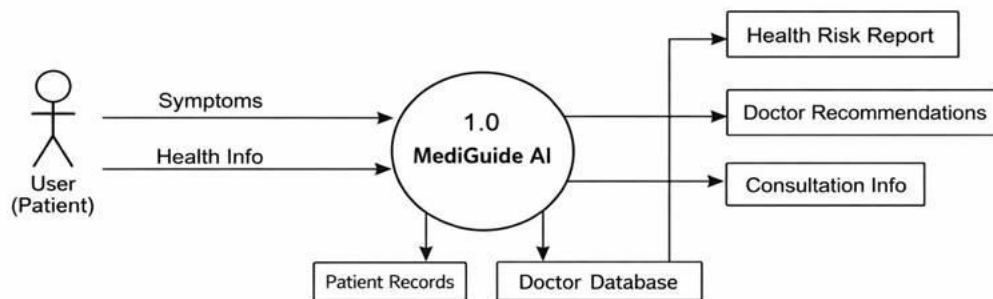


Figure 3.3.1 Data Flow Diagram level 0

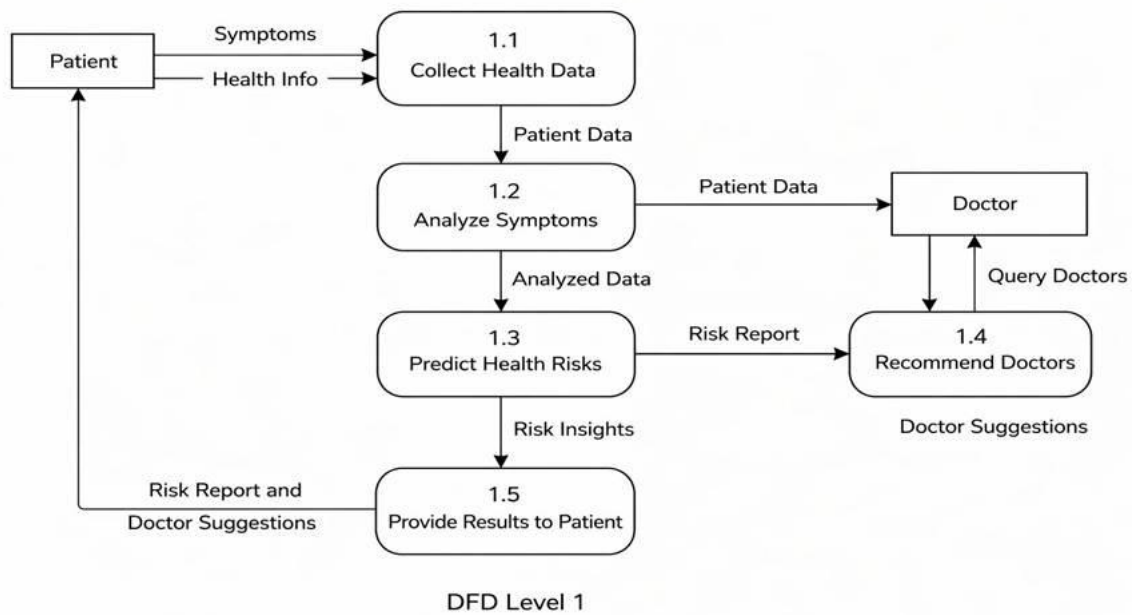


Figure 3.3.2 Data Flow Diagram level 1

3.3.2 Activity Diagram

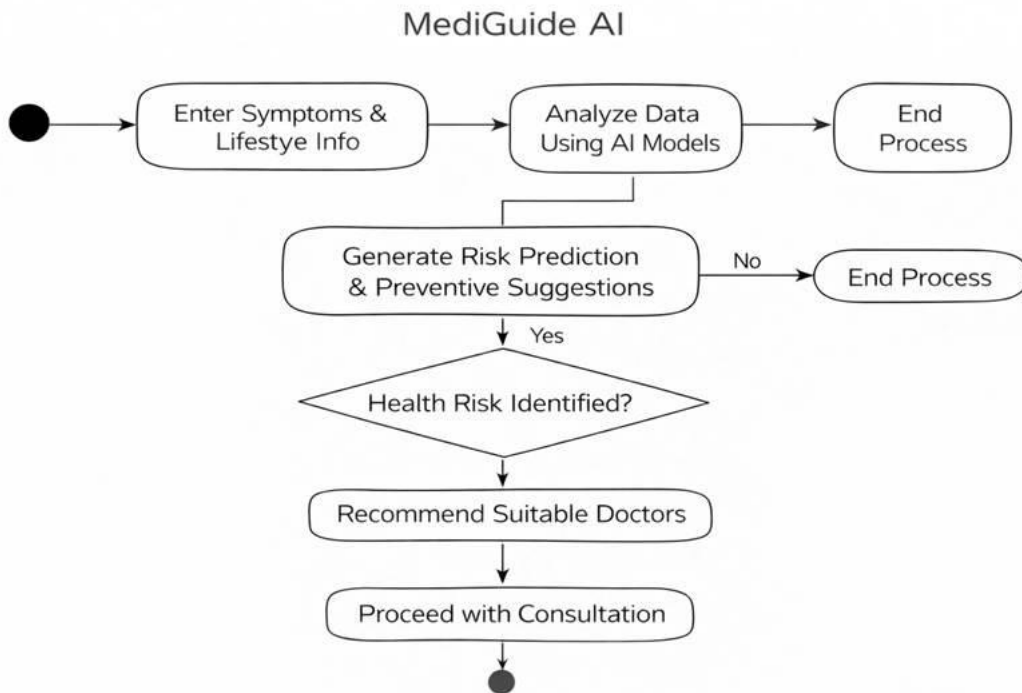


Figure 3.3.3 Activity Diagram

3.3.3 Flow Chart

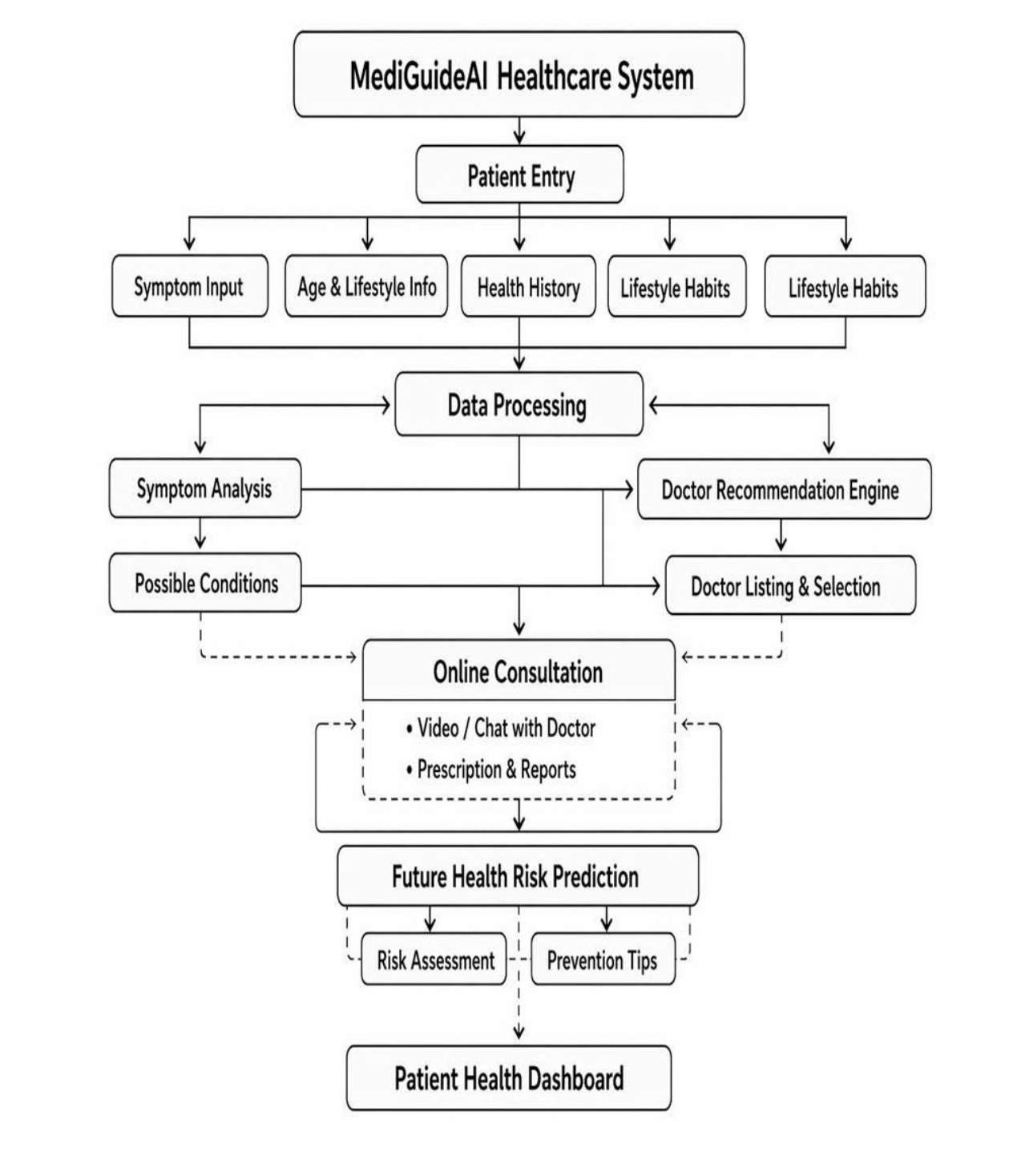


Figure 3.3.4 Flow Chart for Mediguide AI

3.3.4 Class Diagram

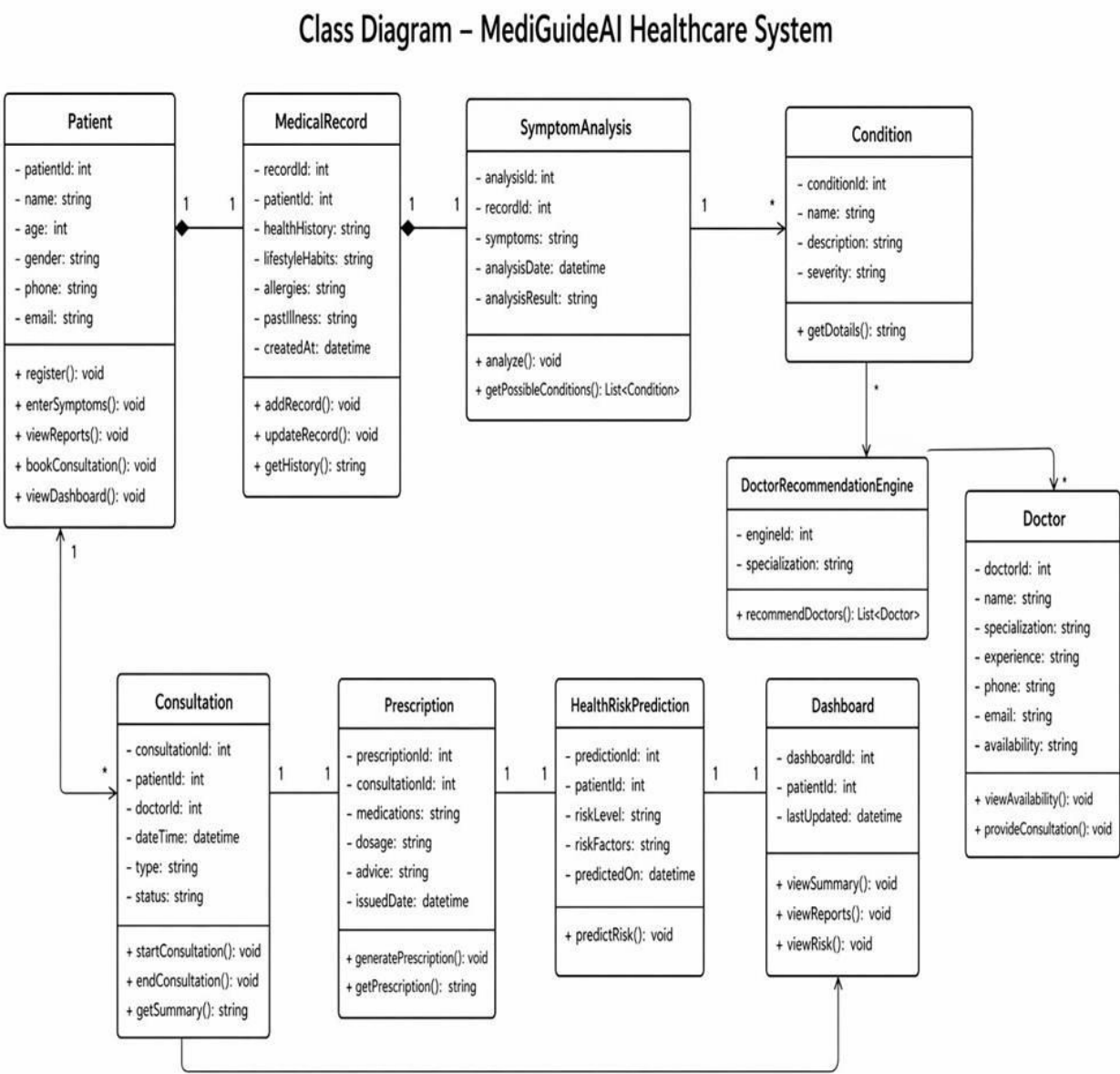


Figure 3.3.5 Class Diagram

3.3.5 ER Diagram

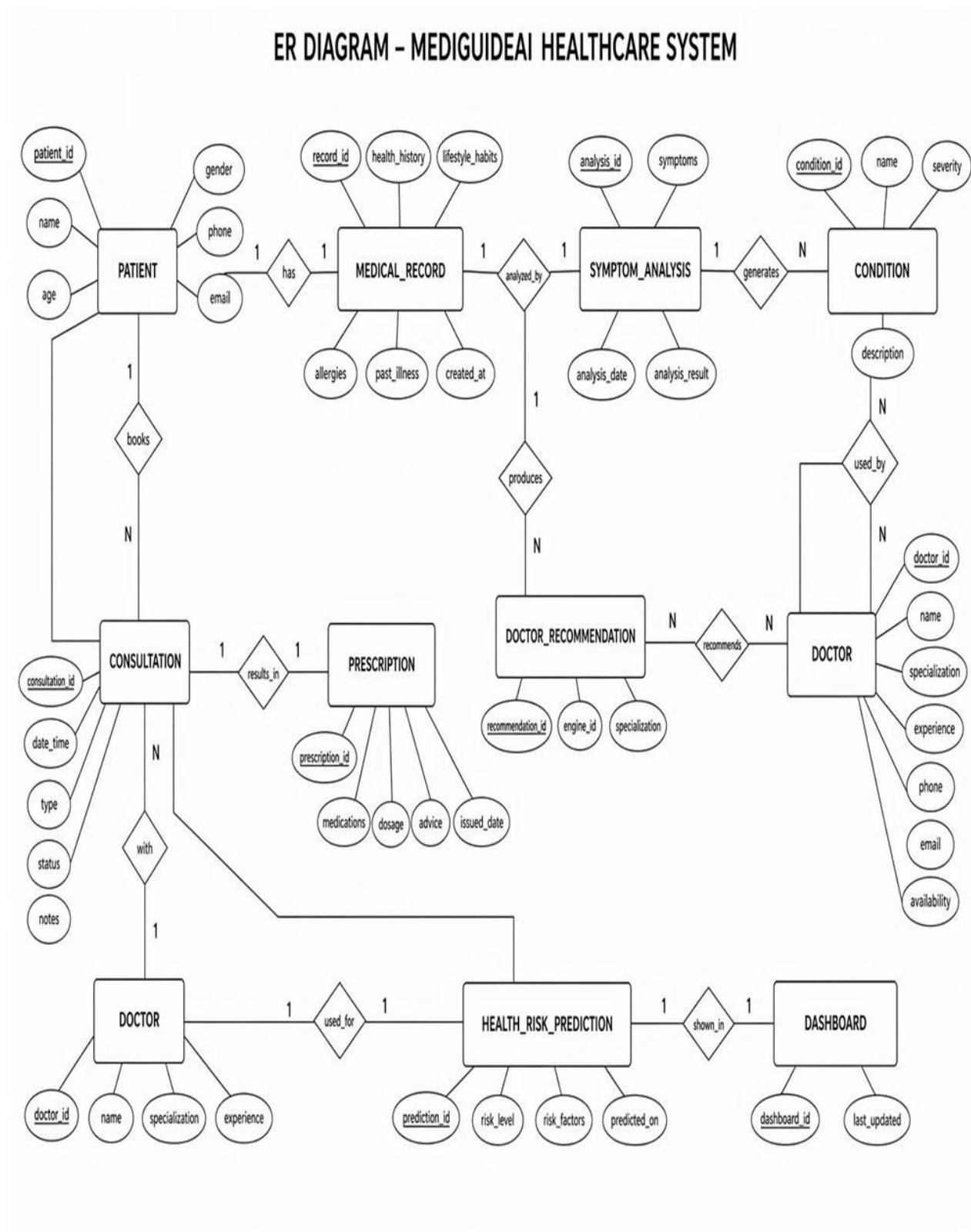


Figure 3.3.6 ERDiagram

3.3.4 Sequence Diagram

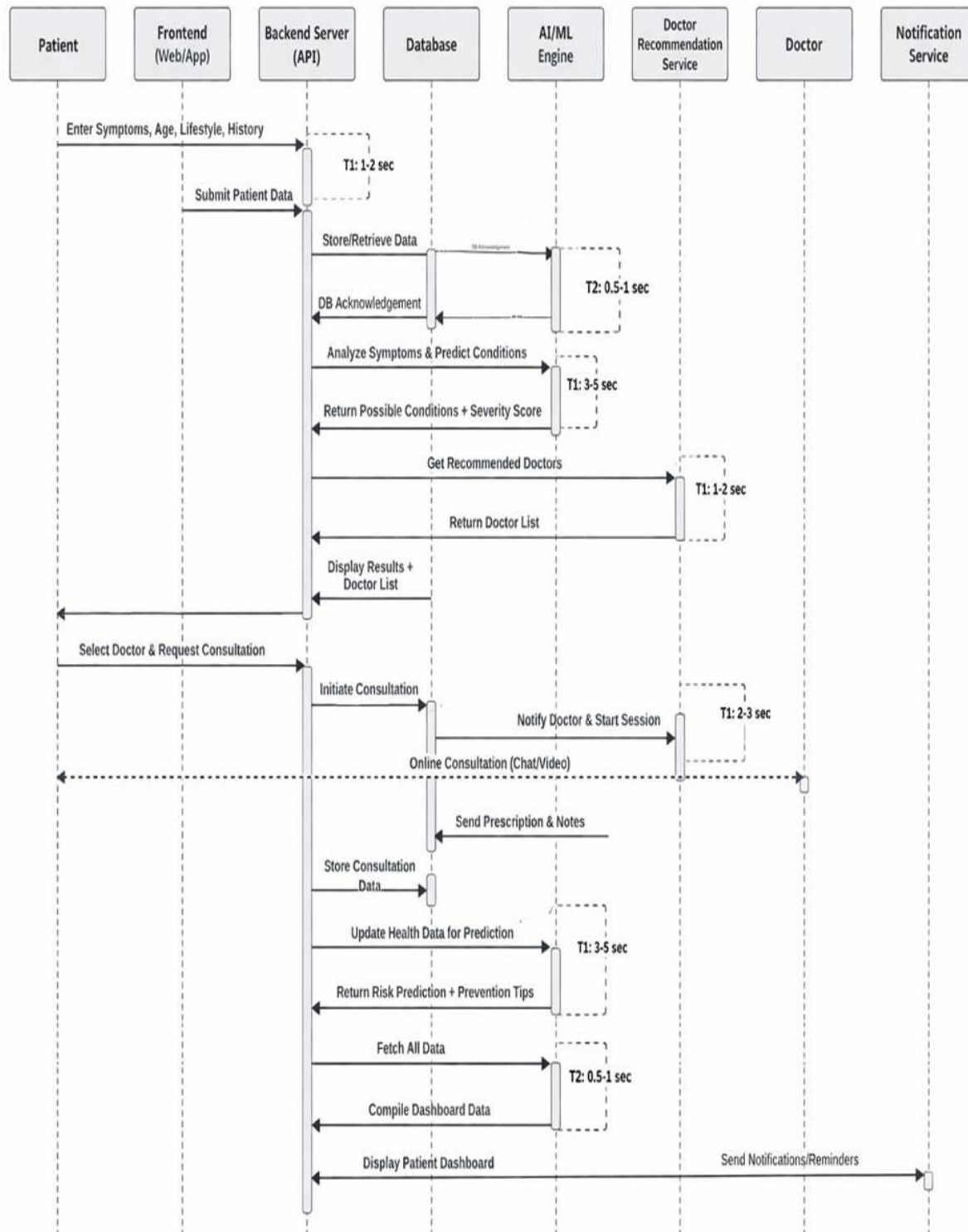


Figure 3.3.7 SequenceDiagram

3.4 Database Design

Database design is one of the most important parts of the MediGuide AI system because it is used to store patient information, doctor details, prediction records, consultation history, and administrator data. A well-structured database helps in efficient data storage, quick retrieval, easy updating of records, and secure handling of sensitive healthcare information.

For the MediGuide AI project, MongoDB is used as the database management system because it is flexible, scalable, and suitable for storing JSON-based data generated by modern web applications. The database is designed in such a way that future features can also be added easily.

3.4.1 Logical Database Design

Logical database design represents the structure of the database in terms of entities, attributes, and relationships. It focuses on what data is stored and how different entities are related to each other.

The main entities identified in the MediGuide AI system are User, Doctor, Prediction, Consultation, and Admin.

User Entity

This entity stores all registered patient details such as User ID, Name, Email, Password, Age, Gender, Contact Number, Address, Medical History, and Lifestyle Information.

Doctor Entity

This entity stores doctor-related information such as Doctor ID, Doctor Name, Specialization, Experience, Availability, Rating, and Contact Details.

Prediction Entity

This entity stores AI-generated health predictions including Prediction ID, User ID, Symptoms, Risk Level, Risk Percentage, Preventive Suggestions, and Prediction Date.

Consultation Entity

This entity stores appointment and chat consultation records such as Consultation ID, User ID, Doctor ID, Date and Time, Consultation Status, and Chat History.

Admin Entity

This entity stores administrator information such as Admin ID, Username, Password, and Role.

Relationships between Entities

- One user can have multiple predictions.
- One user can book multiple consultations.
- One doctor can attend multiple consultations.
- Admin manages users, doctors, and system records.

3.4.2 Physical Database Design

Physical database design defines how the logical database is actually implemented in the database management system. It includes collection names, storage format, indexing methods, and security measures.

For this project, MongoDB database named **mediguide_ai** is used. The following collections are created:

- users
- doctors
- predictions
- consultations
- admins

users Collection

This collection stores all patient registration details, login credentials, personal profile, medical history, and lifestyle information.

doctors Collection

This collection stores doctor profiles, specialization, experience, availability timings, and ratings.

predictions Collection

This collection stores AI-generated risk prediction reports along with user ID reference, symptoms, risk percentage, and preventive suggestions.

consultations Collection

This collection stores booking records, appointment schedules, doctor-patient chat history, and consultation status.

admins Collection

This collection stores administrator login credentials and management roles.

Indexing

Indexes are created on important fields for faster performance:

- Email field in users collection for quick login.
- Specialization field in doctors collection for faster doctor search.
- User ID field in predictions collection for retrieving patient history.
- Date and time field in consultations collection for scheduling.

Security Measures

The following security techniques are applied:

- Passwords are stored in encrypted form using hashing.
- JWT authentication is used for secure sessions.
- HTTPS protocol is used for secure communication.
- Role-based access is provided for administrators.

Backup and Recovery

Regular backups are maintained to prevent data loss. Cloud backup and replication techniques can be used for recovery during server failure.

CHAPTER 4

Implementation, Testing, and Maintenance

4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation

The MediGuide AI project is developed as a modern web-based healthcare platform using advanced programming languages, frameworks, databases, and AI tools. The selected technologies help in building a secure, scalable, and user-friendly system.

Programming Languages Used

JavaScript is used as the primary programming language for both frontend and backend development. It enables smooth integration between client-side and server-side components.

HTML is used to design the structure of web pages.

CSS is used for styling and visual presentation of the user interface.

Frameworks and Libraries Used

React.js / Next.js is used for frontend development to create responsive and interactive user interfaces.

Tailwind CSS is used for fast and modern UI styling.

Node.js is used as the runtime environment for backend development.

Express.js is used to create REST APIs and manage server-side logic.

Database Used

MongoDB is used as the NoSQL database to store user data, doctor details, predictions, consultations, and reports.

AI Technology Used

OpenAI API / Gemini API is used for symptom analysis, predictive health risk assessment, and preventive suggestions.

IDE and Development Tools

Visual Studio Code (VS Code) is used as the main code editor for writing and managing project files.

Postman is used for testing APIs.

Git & GitHub are used for version control and project collaboration.

MongoDB Compass is used for database management and monitoring.

Browser Support

The system can run on modern browsers such as Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari.

4.2 Testing Techniques and Test Plans (According to Project)

Testing is performed to ensure that the MediGuide AI system works correctly, securely, and efficiently. Different testing methods are applied during development.

Testing Techniques Used

1. Unit Testing

Individual modules such as login, registration, prediction module, booking system, and dashboard are tested separately.

2. Integration Testing

Frontend, backend, database, and AI API connections are tested together to ensure proper communication.

3. System Testing

The complete system is tested as a whole to verify all features work correctly.

4. Functional Testing

Testing is done to check whether each function performs according to requirements.

Examples:

- User registration works properly
- Login authentication is correct
- Prediction results are generated
- Doctor recommendation works
- Appointment booking is successful

5. Performance Testing

The system is tested under multiple users to check speed, response time, and server handling capacity.

6. Security Testing

Testing is done for password protection, unauthorized access prevention, session management, and secure API usage.

Test Plan

Test Case ID	Test Scenario	Expected Result
TC01	User Registration	Account created successfully
TC02	User Login	Valid login access granted
TC03	Invalid Login	Error message displayed
TC04	Submit Symptoms	Data accepted successfully
TC05	Generate Prediction	Risk result displayed
TC06	Doctor Search	Suitable doctors shown
TC07	Book Appointment	Booking confirmed
TC08	Logout	Session ended successfully

4.3 Installation Instructions

To install and run the MediGuide AI project, the following steps should be followed:

Software Requirements

- Node.js installed
- MongoDB installed or MongoDB Atlas account
- Visual Studio Code
- Internet connection
- API key for OpenAI / Gemini

Installation Steps

1. Download or clone the project source code from GitHub.
2. Open the project folder in Visual Studio Code.
3. Install required packages using command:

`npm install`

4. Configure environment variables such as MongoDB URI and API keys in `.env` file.
5. Start backend server using command:

`npm run server`

6. Start frontend using command:

`npm run dev`

7. Open browser and run project on:

`http://localhost:3000`

4.4 End User Instructions

The MediGuide AI system is simple and user-friendly. Users can follow these instructions:

For Patients

1. Open the website in browser.
2. Register a new account or login with existing credentials.
3. Fill in health details such as age, symptoms, lifestyle, and medical history.
4. Click on **Analyze** button.
5. View predicted health risk level and preventive suggestions.
6. Check recommended specialist doctors.
7. Book appointment or start consultation chat.
8. View previous reports and history on dashboard.

9. Logout after use.

For Doctors

1. Login using doctor credentials.
2. View patient requests and appointments.
3. Provide consultation through chat.
4. Update appointment status.

For Admin

1. Login through admin panel.
2. Manage users and doctors.
3. Monitor system performance.
4. Maintain database records and backups.

CHAPTER 5

Results and Discussions

5.1 User Interface Representation

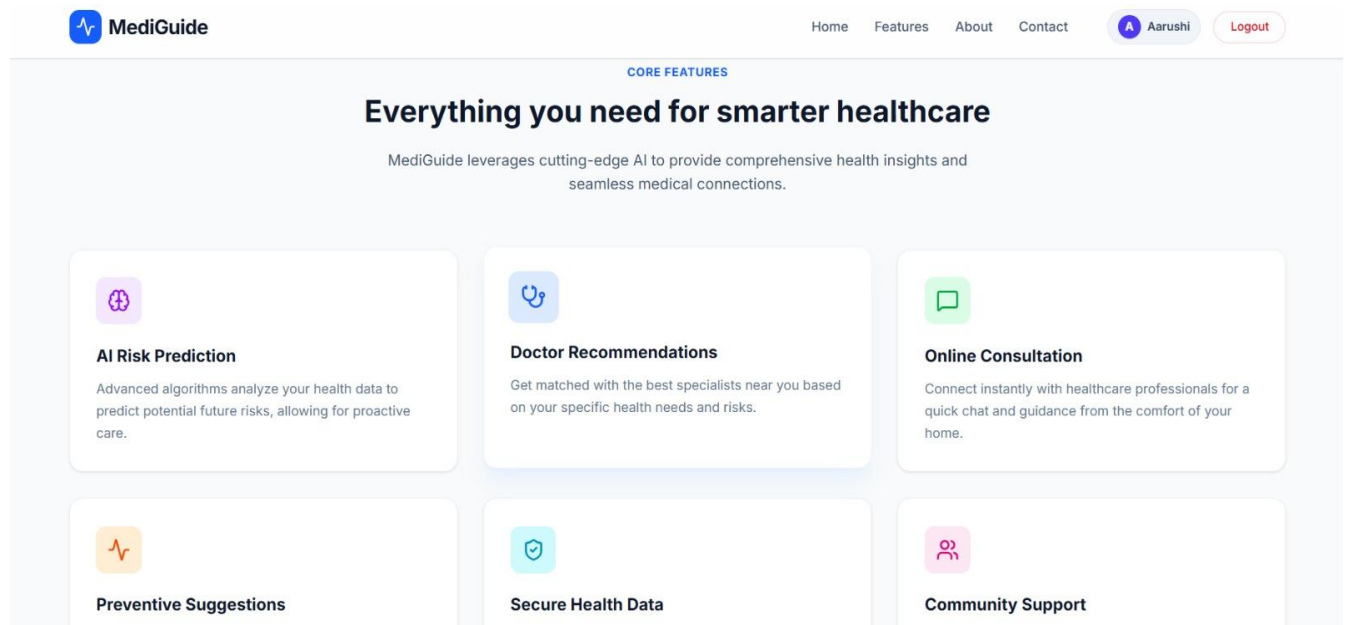
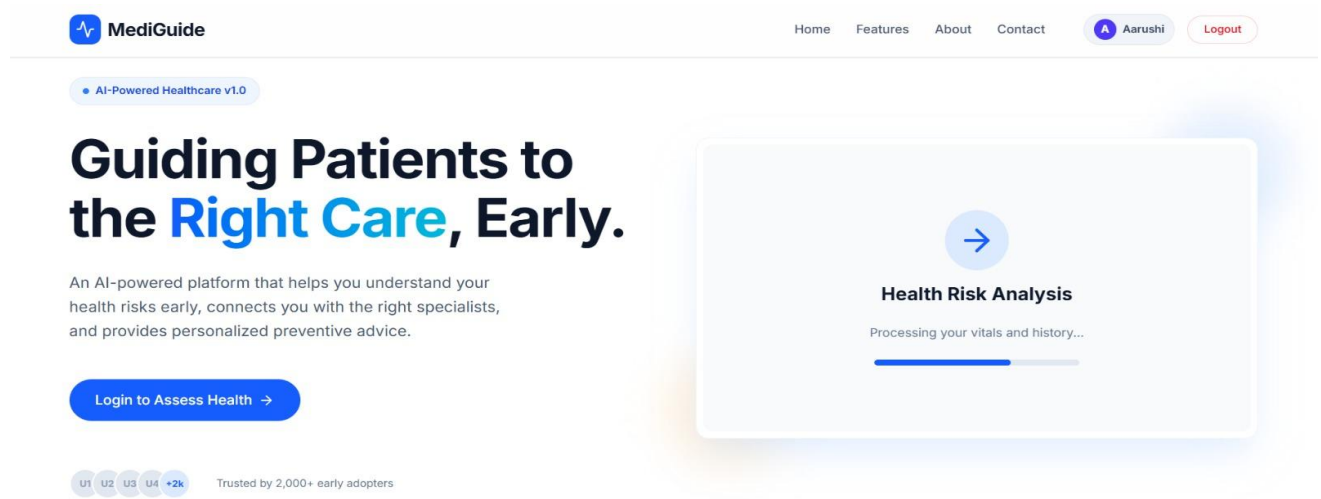
The MediGuide AI system provides a simple and user-friendly interface including Home, Login/Register, Patient Dashboard, Symptom Input, Prediction Results, Doctor Recommendation, Consultation Chat, and Admin Panel.

5.2 Modules of the System

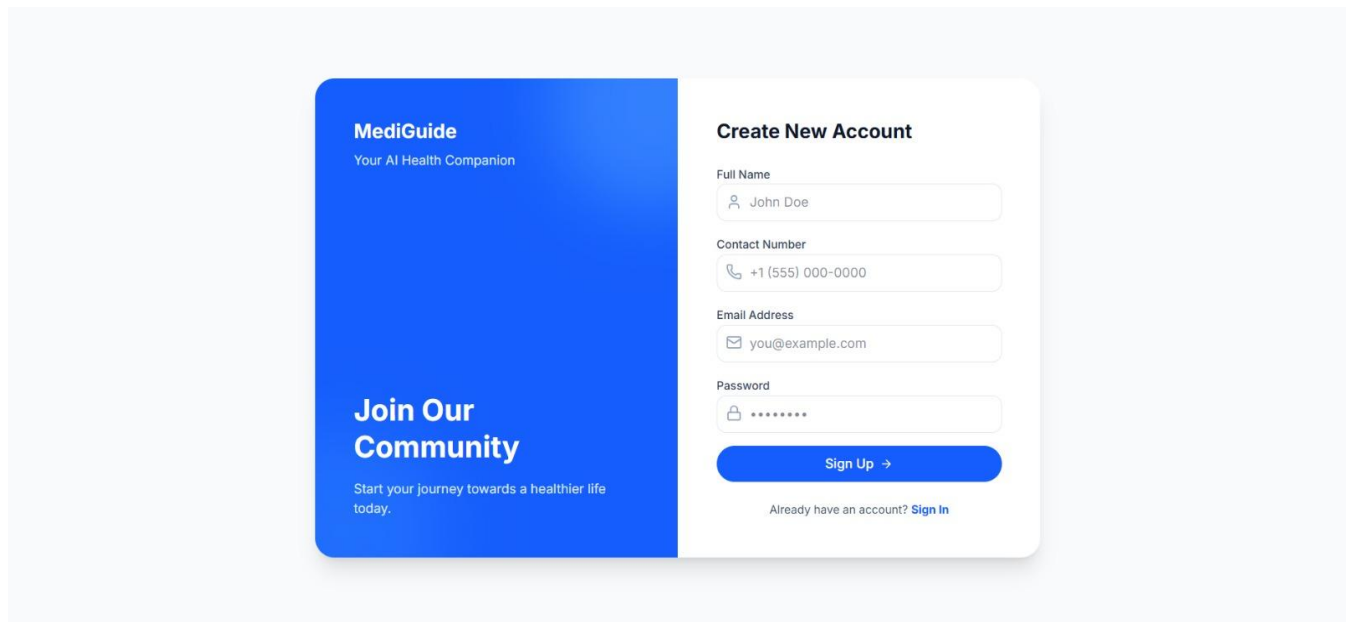
- **Authentication Module:** Login/Register
- **Patient Input Module:** Collects health data
- **AI Prediction Module:** Predicts health risks
- **Doctor Module:** Suggests specialists
- **Consultation Module:** Booking & chat
- **Dashboard Module:** Shows history & reports
- **Admin Module:** Manages system

5.3 Snapshots of System

- Home Page



- Login/Register Page



- Health Form

MediGuide Home Features About Contact Aarushi Logout

Health Assessment Step 5 of 5

Basic Health Symptoms History Lifestyle **Review**

Review
Review all your information before we generate your personalized health report.

Review Your Details

Vitals		Lifestyle	
Age	100	Sleep	9.5 Hrs/Night
Gender	Female	Stress	Moderate
Height	165 Cm	Activity	1 Days/Week
Weight	55 Kg	Diet	Vegetarian

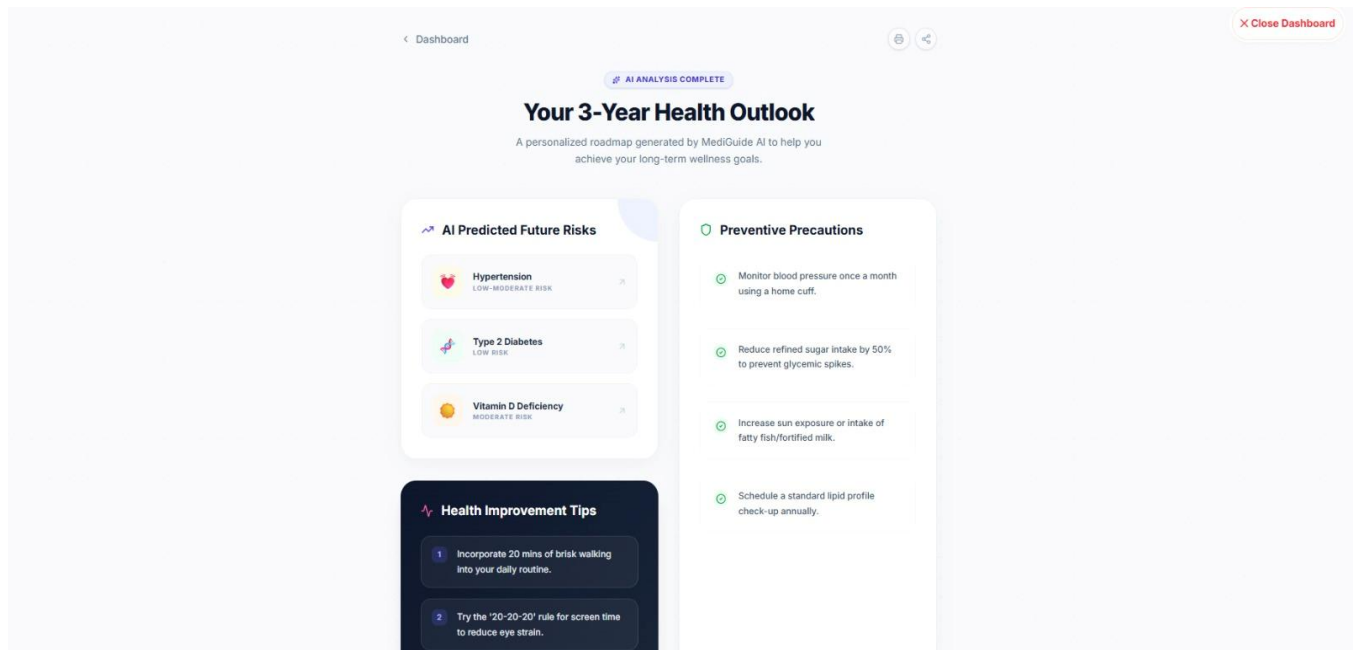
Symptoms & History

Vomiting Body Pain

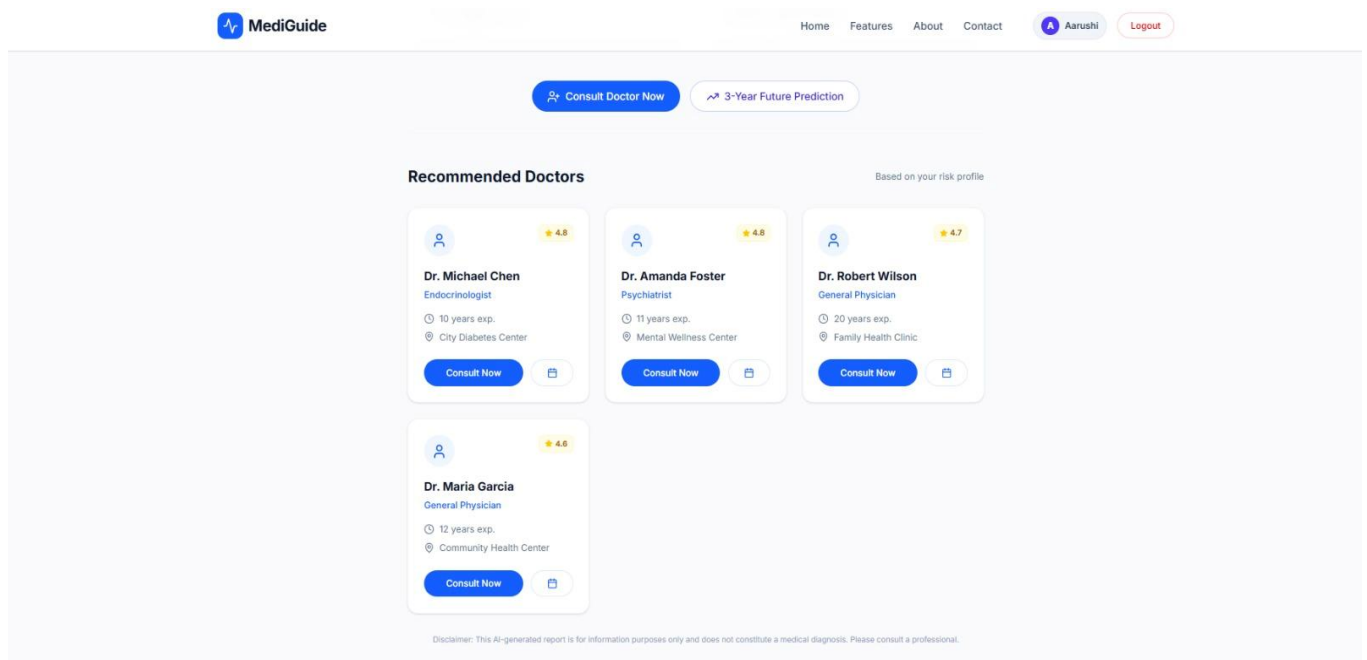
Conditions
None reported

← Back **Predict Health Risks** →

- Prediction Result Page



- Doctor Recommendation Page



5.4 Backend Representation

- **Frontend:** React / Next.js
- **Backend:** Node.js + Express
- **Database:** MongoDB
- **AI:** OpenAI / Gemini API

- **Security:** JWT, Encryption

5.5 Database Snapshots

- **users:** Stores patient details
- **doctors:** Stores doctor info
- **predictions:** Stores risk results
- **consultations:** Stores appointments
- **admins:** Stores admin data

CHAPTER 6

Summary and Conclusions

6.1 Summary

The MediGuide AI project is a web-based healthcare assistance system developed to provide predictive health analysis and doctor recommendations using modern technologies and Artificial Intelligence. The system integrates frontend technologies such as React.js / Next.js, backend technologies like Node.js and Express.js, and a NoSQL database MongoDB. AI capabilities are implemented using APIs such as OpenAI API or Gemini API.

The system allows users to enter their health details, including symptoms, lifestyle habits, and medical history. Based on this data, the AI model predicts potential health risks for the future and provides preventive suggestions. It also recommends suitable medical specialists and enables online consultation features.

The project follows a structured development approach, including requirement analysis, system design, implementation, testing, and deployment. Various modules such as authentication, prediction, doctor recommendation, consultation, dashboard, and admin management are successfully integrated to create a complete healthcare solution.

6.2 Conclusions

The MediGuide AI system successfully demonstrates how Artificial Intelligence can be used in the healthcare domain to promote preventive care and early risk detection. The system provides an efficient platform for users to understand their health conditions and take necessary actions before problems become serious.

The project achieves its main objectives by:

- Providing accurate and quick health risk predictions
- Improving accessibility to medical guidance
- Reducing confusion in selecting appropriate doctors

- Encouraging preventive healthcare practices
- Integrating multiple healthcare services into a single platform

Although the system provides valuable insights, it is important to note that it is not a replacement for professional medical diagnosis. The predictions are based on user input and AI models, so accuracy depends on the quality of data provided.

CHAPTER 7

7.1 Future Scope

The MediGuide AI system can be further enhanced with the following improvements:

- Integration with real hospital databases and Electronic Health Records (EHR)
- Implementation of advanced Machine Learning models for higher accuracy
- Addition of voice-based interaction and chatbot support
- Mobile application development for better accessibility
- Real-time doctor availability and video consultation features
- Wearable device integration for continuous health monitoring

7.2 Appendix

The appendix section includes additional supporting materials used in the project. These are not part of the main report but provide extra information for better understanding.

The appendix may include:

- Sample input forms used for data collection
- Screenshots of system interfaces
- API request and response formats
- Database schema details
- Source code snippets
- Test case documents

7.3 Bibliography

The bibliography lists all the sources of information, research papers, websites, and tools referred to during the development of the project.

Books and Research Papers

- [1] IEEE, *Research Papers on Artificial Intelligence in Healthcare*, IEEE Xplore Digital Library, 2020–2025.
- [2] Springer, *Journals on Predictive Analytics and Medical Systems*, SpringerLink, 2019–2025.
- [3] Various Authors, “Machine Learning Applications in Disease Prediction,” *International Journal of Medical Informatics*, 2020–2024.
- [4] Various Authors, “Preventive Healthcare Systems and Digital Health Platforms,” *Journal of Healthcare Engineering*, 2019–2024.
- [5] Various Authors, “AI-Based Medical Systems,” IEEE and Springer Publications, 2020–2025.
- [6] Various Authors, “Telemedicine and Online Consultation Platforms: Case Studies,” *Healthcare Technology Letters*, 2018–2024.

Web References

- [7] OpenAI, “API Documentation,” [Online]. Available: <https://platform.openai.com>
- [8] Google, “Gemini API Documentation,” [Online]. Available: <https://ai.google.dev>
- [9] World Health Organization, “Global Health Reports and Datasets,” [Online]. Available: <https://www.who.int>
- [10] Centers for Disease Control and Prevention, “Healthcare Guidelines,” [Online]. Available: <https://www.cdc.gov>
- [11] MongoDB, “MongoDB Documentation,” [Online]. Available: <https://www.mongodb.com/docs>
- [12] React, “React.js Documentation,” [Online]. Available: <https://react.dev>
- [13] Node.js, “Node.js Documentation,” [Online]. Available: <https://nodejs.org>
- [14] Express.js, “Express.js Documentation,” [Online]. Available: <https://expressjs.com>
- [15] Next.js, “Next.js Documentation,” [Online]. Available: <https://nextjs.org>
- [16] Kaggle, “Medical Datasets,” [Online]. Available: <https://www.kaggle.com>

[17] Various Authors, “Technical Blogs on MERN Stack Development,” Medium, Dev.to, 2020–2025.

Tools and Technologies

[18] Visual Studio Code, “Documentation,” [Online]. Available: <https://code.visualstudio.com>

[19] GitHub, “Documentation,” [Online]. Available: <https://docs.github.com>

[20] Postman, “API Testing Tool Documentation,” [Online]. Available: <https://www.postman.com>

[21] MongoDB Compass, “Database Management Tool,” [Online]. Available: <https://www.mongodb.com/products/compass>

[22] Git, “Version Control Documentation,” [Online]. Available: <https://git-scm.com>

Additional References

[23] Coursera, “Courses on AI and Web Development,” [Online].

[24] Udemy, “AI and Web Development Tutorials,” [Online].

[25] YouTube, “Educational Content on AI and MERN Stack,” [Online].

[26] Various Authors, “RESTful APIs and JSON Data Handling,” Technical Documentation and Articles, 2020–2025.

[27] Various Authors, “Healthcare Awareness and Preventive Medicine,” WHO and Public Health Articles.

[28] Various Authors, “AI in Healthcare Implementation Discussions,” Stack Overflow, Reddit, and Developer Forums.